

Polymorphism

The Many Faces of Objects

Mastering Compile-Time & Run-Time Dispatch

OVERLOADING • VIRTUAL • DYNAMIC BINDING

1. One Interface, Multiple Actions

The word **Polymorphism** comes from the Greek words poly (many) and morph (forms). In C++, it allows a single function name or operator to have different meanings depending on the context.

The Core Advantage:

It allows you to perform a single action in different ways. It decouples "what" you want to do from "how" it's specifically implemented for a certain type.

The Real-world Analogy:

Think of a person. That person can be a student, an employee, and a citizen simultaneously. Their behavior changes depending on the environment (context) they are in.

2. Compile-Time Resolve

In **Static Polymorphism**, the compiler decides which function to execute during the compilation process. It is extremely fast because there is no searching happening during the actual running of the program.

Achieved via:

- **Function Overloading:** Same name, different parameter lists.
- **Operator Overloading:** Customizing standard symbols like + or -.
- **Templates:** Generic programming (Advanced).

3. Function Overloading

The compiler looks at the **arguments** you pass to decide which version of a function to call.

```
class Printer {
public:
    // Form 1: Int
    void print(int i) { cout << "Int: " << i; }

    // Form 2: String
    void print(string s) { cout << "String: " << s; }
};

int main() {
    Printer p;
    p.print(10);          // Compiled to use Form 1
    p.print("Hello");     // Compiled to use Form 2
}
```

4. Operator Overloading

You can teach C++ how to use standard operators (like +) with your custom classes. This makes your classes behave like built-in primitive types.

```
class Box {
public:
    int weight;
    Box(int w) : weight(w) {}

    // Tell '+' how to handle two Boxes
    Box operator+(const Box& b) {
        return Box(this->weight + b.weight);
    }
};

int main() {
    Box b1(10), b2(20);
    Box b3 = b1 + b2; // Logical and intuitive syntax
}
```

5. Run-Time Resolve

In **Dynamic Polymorphism**, the decision of which function to call is delayed until the program is actually running. This provides the ultimate flexibility.

The Requirement:

To use run-time polymorphism, you must use **Inheritance** and **Pointers/References** of the Base class type.

The Mechanics:

It uses a "Virtual Table" (vtable) to look up the correct function at the moment of execution.

6. The 'virtual' Keyword

A virtual function in the Base class tells the compiler: "I might be overridden by a child. Wait until runtime to check for the most specific version."

```
class Base {  
public:  
    virtual void show() { cout << "Base"; }  
};  
  
class Derived : public Base {  
public:  
    void show() override { cout << "Derived"; }  
};
```

The `override` keyword is technically optional but highly recommended to prevent spelling mistakes that create new functions instead of overriding.

7. Polymorphic Pointers

The magic happens when a Base pointer points to a Child object.

```
int main() {
    Base* ptr;
    Derived d;
    ptr = &d; // Upcasting

    // Even though ptr is a "Base" pointer,
    // it finds the "Derived" show() function
    ptr->show();
}
```

Without the `virtual` keyword, `ptr->show()` would always call the Base version, ignoring the true identity of the object.

8. Dynamic Course Dispatch

```
class Course {  
public:  
    virtual void getInfo() { cout << "Generic Course"; }  
};  
  
class CppCourse : public Course {  
public:  
    void getInfo() override { cout << "C++ Masterclass"; }  
};  
  
class JavaCourse : public Course {  
public:  
    void getInfo() override { cout << "Java Bootcamp"; }  
};
```

A single list of `Course*` pointers could contain C++ and Java courses, each behaving correctly when `getInfo()` is called.

9. The Safety Rule

If you have any virtual functions in your class, you **must** make your destructor virtual. Failure to do so leads to memory leaks when deleting child objects via base pointers.

```
class Base {  
public:  
    virtual ~Base() { /* Clean up Base */ }  
};
```

Rule of Thumb: Always add `virtual ~ClassName() = default;` if you plan to use inheritance.

10. Comparative Analysis

How do you choose between the two types of polymorphism?

Feature	Static	Dynamic
Decision Time	Compile Time	Run Time
Performance	Faster (No vtable)	Slightly Slower
Keyword	None (Overloading)	Virtual
Flexibility	Rigid	Highly Flexible

11. Abstract Blueprints

Sometimes a Base class shouldn't have any implementation. It only defines an interface. This is a **Pure Virtual Function**.

```
class Shape {  
public:  
    // Pure virtual function  
    virtual void draw() = 0;  
};
```

This makes the class "Abstract." You cannot create an object of type Shape; you can only create objects of its children that implement draw().

12. Why Polymorphism Matters

1. Extensibility

You can add a `PythonCourse` to your system tomorrow without changing any of the code that handles a list of `Course*` objects.

2. Simplified Interfaces

The user only needs to call `process()`. Polymorphism ensures the right kind of processing happens for the specific object type.

3. Modern C++ Standard

Polymorphism is the foundation of Design Patterns and standard frameworks used in game engines (Unreal) and GUI libraries (Qt).

13. Final Recap

- **Polymorphism:** One name, different behaviors.
- **Overloading:** Compile-time choice (Static).
- **Virtual:** Run-time choice (Dynamic).
- **Pointer Use:** Needed for dynamic behavior.
- **Abstract:** Uses `= 0` to enforce child logic.

Skill Lab: The Animal Kingdom

Create an abstract base class `Animal` with a pure virtual `makeSound()`. Create children `Lion` and `Bird`. Use a pointer array of `Animal*` to store both and call their sounds in a loop.

LOGIC TRANSFORMED: POLYMORPHISM MASTERED